GitHub Setup for DATA 3402

WSL, macOS and Linux — do this once, use it all semester

Amir Farbin

Contents

GitHub Setup for DATA 3402

WSL, macOS and Linux. Do this once; the same workflow carries the whole semester.

What you are setting up

You submit every lab in this course through GitHub. The arrangement is:

- You **fork** the course repository, giving you your own copy on your own account.
- You **clone your fork** onto your machine.
- You wire the clone so that **`git pull` pulls from the class** and **`git push` pushes to your fork**.
- You submit by committing your notebook and pushing.

One line to remember: **pull from class, push to my fork, never force push.**

Because the course repository is public, your fork will be public too. That is expected.

Where to get help. Bring anything that doesn't work to the Lab 4 session — that session exists partly to debug this. You are not expected to have it working perfectly on your own first.

Step 0 — prerequisites

Windows: Ubuntu installed through WSL, and a terminal that opens into Ubuntu.

macOS: Terminal, with the command line tools installed:

```
xcode-select --install
```

Linux: you already have what you need.

Everyone: a GitHub account.

Step 1 — fork the course repository

In a browser, log in to GitHub and go to the course repository:

https://github.com/UTA-DataScience/DATA3402.Fall.2026

Press **Fork** at the top right and fork it into your own account. You should end up looking at your own copy, at `github.com/YOUR_USERNAME/DATA3402.Fall.2026`.

Step 2 — authenticate

Passwords no longer work for git over HTTPS. Pick one of these. **Option A is recommended** and is what most of you should do.

Option A — GitHub CLI (recommended)

Works the same on WSL, macOS and Linux, and needs no keys.

1. Install the GitHub CLI:
   - Ubuntu / WSL: `sudo apt install gh`
   - macOS: `brew install gh`, or download the installer from https://cli.github.com
2. Log in:

   ```
   gh auth login
   ```

   Choose **GitHub.com**, then **HTTPS**, then **authenticate via browser**. Approve the login in the browser that opens — a passkey works here.
3. Check it:

   ```
   gh auth status
   ```

That's it. Ordinary `git` commands now work over HTTPS.

Option B — SSH keys

More setup, and worth it if you already know SSH or expect to use several machines.

1. Generate a key:

   ```
   ssh-keygen -t ed25519 -C "your_netid@uta.edu"
   ```

   Press Enter at every prompt to accept the defaults.

2. Start the agent and add the key:

   ```
   eval "$(ssh-agent -s)"
   ssh-add ~/.ssh/id_ed25519
   ```

3. Copy the public half:

   ```
   cat ~/.ssh/id_ed25519.pub
   ```

   On GitHub go to **Settings → SSH and GPG keys → New SSH key**, paste it, and save.

4. Test:

   ```
   ssh -T git@github.com
   ```

   It should tell you that you have authenticated.

Whichever you pick, this is a one-time setup per machine. If authentication stops working later, rerun `gh auth login` (Option A) or check that your key is still added (Option B).

Step 3 — make a course directory

```
cd ~
mkdir Data-3402
cd Data-3402
```

> **WSL users, this matters.** Work inside your Linux home directory, as above — *not* under `/mnt/c/...`. Working on the Windows filesystem from WSL is much slower and causes git and Jupyter to misbehave in ways that are hard to diagnose.

Step 4 — clone your fork

On **your fork's** GitHub page, press the green **Code** button and copy the URL. Then, matching the method you chose in Step 2:

```
# Option A (HTTPS)
git clone https://github.com/YOUR_USERNAME/DATA3402.Fall.2026.git

# Option B (SSH)
git clone git@github.com:YOUR_USERNAME/DATA3402.Fall.2026.git

cd DATA3402.Fall.2026
```

With SSH the first connection asks *"Are you sure you want to continue connecting?"* — answer `yes`.

Step 5 — tell git who you are

If git complains that your **author identity is unknown**, set it:

```
git config --global user.name "Your Full Name"
git config --global user.email "your_netid@uta.edu"
git config --global --list
```

Step 6 — fix the remotes

This is the step that makes the whole arrangement work, and the one people skip.

Your clone currently points at your fork for everything. You want it to *fetch from the class* and *push to your fork*.

```
git remote -v                        # see what you have now
git remote remove origin
git remote add origin https://github.com/UTA-DataScience/DATA3402.Fall.2026.git
```

Then point pushes at your fork — the URL you cloned from:

```
# Option A (HTTPS)
git remote set-url --push origin https://github.com/YOUR_USERNAME/DATA3402.Fall.2026.git

# Option B (SSH)
git remote set-url --push origin git@github.com:YOUR_USERNAME/DATA3402.Fall.2026.git
```

Check it:

```
git remote -v
```

You want **fetch → the class repository** and **push → your fork**. If both lines say the same thing, the `set-url --push` didn't take; run it again.

There is one remote, called `origin`, and it is deliberately lopsided. `git pull` brings you this week's material; `git push` sends your work to your fork.

Step 7 — your first push

```
git add .
git commit -m "Initial setup"
git push
```

If git says **the current branch main has no upstream branch**:

```
git push --set-upstream origin main
```

If it says **rejected (non-fast-forward)** — which is common and expected the first time, because your fork and the class repo have separate histories:

```
git config pull.rebase false
git pull origin main --allow-unrelated-histories
git push
```

That is safe.

Every week after that

```
git pull                              # get the new material
# ... work on your notebook ...
git add .
git commit -m "Lab 3 solution"
git push                              # goes to your fork
```

Name your solution differently from the file we hand out. If the lab arrives as `Lab.3.ipynb`, work in `Lab.3.solution.ipynb`:

```
mv Lab.3.ipynb Lab.3.solution.ipynb
```

If you edit the handed-out file in place, the next `git pull` will try to merge our changes into yours and you will get a merge conflict in a Jupyter notebook — which is genuinely unpleasant, because the conflict lands in the notebook's JSON rather than in your code. See `Labs/DATA3402_Lab3_Merge_Conflict_Guide.pdf` if it happens anyway.

When something breaks

It says	What it means	What to do
`not a git repository`	you're in the wrong directory	`cd ~/Data-3402/DATA3402.Fall.2026`
`author identity unknown`	name and email not set	Step 5
`permission denied (publickey)`	SSH key not registered	Step 2, Option B — or switch to Option A
`rejected (non-fast-forward)`	the remote has commits you don't	`git pull origin main --allow-unrelated-histories`
`divergent branches`	git wants a merge strategy	`git config pull.rebase false`
`Authentication failed` over HTTPS	password auth, which is gone	`gh auth login`
push went to the class repo	`set-url --push` didn't apply	redo Step 6 and check `git remote -v`

Final check

```
git status
git branch -vv
git remote -v
```

If `git remote -v` shows fetch pointing at `UTA-DataScience` and push pointing at your own account, you are set for the semester.